SCALABILITY & RELIABILITY DESIGN

A. 10,000 REQUESTS / 100 UNITS - EXACTLY WHAT HAPPENS

1. EDGE
   CDN/WAF serves static content, blocks bots, applies per-IP/user rate limits. Waiting room meters Buy traffic into the origin (e.g. 2-5k rps).

2. GATEWAY
   Verifies JWT, requires Idempotency-Key, checks the in-memory sold-out flag (pushed via pub/sub). Once flagged, ~all remaining requests get 409 SOLD_OUT here, never touching Redis/DB.

3. STOCK GATE (the exact contention point)
   One Redis Lua script per request, atomic and single-threaded per key:

```
-- KEYS: {sku}:stock, {sku}:idem:<key>, {sku}:buyer:<user>
-- (hash tag {sku} = same slot)
local dup = redis.call('GET', KEYS[2]); if dup then return {'DUP', dup} end
if redis.call('EXISTS', KEYS[3]) == 1 then return {'LIMIT'} end
local s = tonumber(redis.call('GET', KEYS[1]) or '0')
if s <= 0 then return {'SOLD_OUT'} end
redis.call('DECR', KEYS[1])
redis.call('SET', KEYS[2], ARGV[1], 'EX', ARGV[2])
redis.call('SET', KEYS[3], ARGV[1], 'EX', ARGV[2])
return {'OK', ARGV[1]}
```

   Exactly 100 requests get OK; 9,900 get SOLD_OUT; duplicates get the original reservation_id.

4. AUTHORITATIVE WRITE
   Each admitted request runs one Postgres transaction: conditional UPDATE ... WHERE available_quantity >= 1, insert reservation (unique idempotency key), insert outbox. CHECK constraints make overselling impossible even if Redis is wrong.

5. PAYMENT
   ~95 succeed -> PaymentSucceeded -> Order created (unique payment_id) -> OrderConfirmed -> reservation SOLD.
   ~5 fail -> reservation RELEASED -> counter +1 in DB and Redis -> sold-out flag cleared -> waiting buyers can take the units.

6. UNPAID RESERVATIONS
   Expire after TTL via ExpiryWorker (FOR UPDATE SKIP LOCKED), releasing stock.

7. RESULT
   sold + reserved <= 100; one order per paid reservation.

B. SCALING & STATE

- Stateless pods (gateway, services) scale by HPA on CPU/RPS/queue lag; state only in Redis/Postgres/Kafka.
- Redis Cluster (shards + replicas, AOF every second); Postgres primary + sync replica + read replicas for browse; Kafka RF=3 partitioned by reservation_id.
- Browse/catalogue: CDN + Redis cache (cache-aside, short TTL, stampede protection with request coalescing).
- Throttle -> queue -> reject: edge rate limit (reject 429) -> waiting room (queue) -> gateway concurrency limit (reject 503 + Retry-After) -> Kafka (buffer for async work).

C. 50x TRAFFIC (500k rps)

Edge   : Pre-warm CDN, tighten waiting-room admission, aggressive
                   bot challenge; sold-out page served statically.
Gateway/services : Pre-scale pods before sale; HPA headroom; connection
                   pools capped.
Redis   : Origin sees only admitted traffic; stock key is one slot,
                   so scaling = fewer calls, not more shards; per-user keys
                   spread across shards.
Postgres   : Untouched by the surge: only <=100 reservation writes
                   (+ payments).
Kafka   : Add partitions ahead of time; consumer groups scale with
                   lag.
Trade-off   : Some legitimate users wait or are rejected; fairness
                   maintained via waiting-room ordering.

For many SKUs, shard stock by SKU (separate slots). For one mega-SKU, split stock into N buckets (e.g. 10 x 10 units) with randomised bucket choice and fallback scan.

D. FAILURE HANDLING

Payment declined
  PaymentFailed -> reservation RELEASED -> stock returned -> user notified.

Payment timeout
  Payment = UNKNOWN; retry/status-poll with SAME idempotency key; never
  re-charge with a new key; reservation held (grace) until resolved;
  unresolved after grace -> gateway void/refund + release.

Duplicate payment request
  Unique reservation_id / idempotency_key returns existing payment.

Payment success, Order Service down
  PaymentSucceeded is durable in outbox/Kafka; customer sees "payment
  received"; reservation already CONFIRMED (not expirable); order created on
  recovery; after N retries -> DLQ + alert; reconciler matches SUCCEEDED
  payments without orders and replays; last resort: refund (compensation).

Payment success after reservation expired
  Atomic check on callback: reservation RELEASED -> auto refund (or
  re-reserve if stock free).

Postgres primary fails
  Promote sync replica (RPO 0); inventory pods retry; Redis gate keeps
  rejecting excess; writes resume; reconciler rebuilds Redis counter = DB
  available_quantity.

Redis fails/loses data
  Gate replica promoted; if lost, buy path pauses (503), counter rebuilt
  from DB (available), per-buyer keys rebuilt from reservations; DB
  constraints prevent oversell during gap.

Payment gateway down
  Circuit breaker opens; fail fast with "try again" while reservations stay
  valid (TTL extended once); secondary provider via Factory/Strategy if
  configured.

Kafka consumer crash
  Redelivery; idempotent consumers; poison messages -> DLQ.

Redis/DB drift
  Reconciler job every 30 s compares counters; alerts on mismatch.

E. TIMEOUT / RETRY / BREAKER POLICY

- Gateway call timeout 3 s; 2 retries, exponential backoff + jitter, same idempotency key.
- Breaker opens at 50% failures over 20 calls; half-open probe after 10 s.
- Bulkhead: separate thread/connection pool per dependency.
- Consumers: 5 retries with backoff, then DLQ.

F. TRADE-OFFS

- Redis gate + DB = two systems to keep consistent (complexity), in exchange for taking >99% of contention off the database.
- Async order creation = eventual consistency (short "payment received, order confirming" state) in exchange for resilience to Order Service outage.
- Single-region buy path = lower availability on region loss, in exchange for one simple inventory authority.